

Frontend Coordination Notes — Layout, UI States, Auth and Roles

Goal

The frontend work should now be divided clearly:

- **Flor:** API implementation, Axios services, authentication flow, employee/admin registration logic.
- **Teammate:** UI/UX structure, protected layout, header/sidebar, reusable visual components, page shell consistency.

The backend is already protected with JWT, so all protected frontend pages must respect authentication and user roles.

1. Important authentication context

The frontend now uses a refactored authentication system.

Current auth files

```
src/services/api.ts
src/services/authService.ts
src/services/authHelpers.ts
src/routes/ProtectedRoute.tsx
src/routes/AdminRoute.tsx
```

Responsibilities

File	Responsibility
<code>api.ts</code>	Axios instance, token injection, 401 handling
<code>authService.ts</code>	Login/logout logic and storing auth data
<code>authHelpers.ts</code>	Reading auth data, checking roles, clearing session
<code>ProtectedRoute.tsx</code>	Blocks unauthenticated users
<code>AdminRoute.tsx</code>	Blocks non-admin users

2. Axios rule

Do **not** use raw `fetch()` in new components.

Use the centralized Axios client:

```
import api from "@services/api"
```

or, depending on current project alias support:

```
import api from "../services/api"
```

The Axios instance already handles:

- adding `Authorization: Bearer <token>`
- redirecting to `/login` when backend returns `401`
- centralizing the backend base URL through the Vite proxy

Frontend components should not manually add the token.

3. Role rules

Backend roles used by frontend:

Backend role	Frontend meaning
<code>ROLE_ADMIN</code>	Admin / manager user
<code>ROLE_EMPLOYEE</code>	Normal employee user

Use helpers from:

```
src/services/authHelpers.ts
```

Available helpers:

```
getToken()  
getUsername()  
getRoles()  
hasRole(role)  
isAuthenticated()
```

```
isAdmin()  
isEmployee()  
clearAuth()
```

Important

Do not manually check roles like this inside components:

```
localStorage.getItem("roles")
```

Instead use:

```
import { isAdmin, isEmployee, getUsername } from "@services/authHelpers"
```

This keeps role logic centralized.

4. Protected layout task

Create or complete:

```
src/layouts/ProtectedLayout.tsx
```

Expected structure:

```
import { Outlet } from "react-router-dom"  
import Header from "@components/layout/Header"  
import Sidebar from "@components/layout/Sidebar"  
  
export default function ProtectedLayout() {  
  return (  
    <div>  
      <Header />  
      <div>  
        <Sidebar />  
        <main>  
          <Outlet />  
        </main>  
      </div>  
    </div>  
  )  
}
```

```
)  
}
```

The exact styling can change, but the layout should allow all protected pages to share:

- same header
- same sidebar
- same spacing
- same page container
- same responsive behavior

5. Sidebar role visibility

Sidebar items should depend on role.

Example:

Link	Admin	Employee
Dashboard	yes	yes
My Agents	yes	yes
Workflows	yes	yes
Employees	yes	no
Subordinate Agents	yes	no
Settings	yes	yes

Example logic:

```
{isAdmin() && (  
  <NavLink to="/employees">  
    Employees  
  </NavLink>  
)}
```

6. Header requirements

Header should include:

- app/project name

- current username
- current role badge if possible
- logout button
- responsive menu button for mobile/sidebar
- no hardcoded fake user

Use:

```
getUsername()  
isAdmin()  
isEmployee()
```

Logout should call the existing logout logic from `authService.ts`.

7. Reusable UI state components

Please create:

```
src/components/common/LoadingSpinner.tsx  
src/components/common/ErrorMessage.tsx  
src/components/common/EmptyState.tsx
```

LoadingSpinner

Used while a page waits for API data.

Example usage:

```
if (loading) {  
  return <LoadingSpinner />  
}
```

ErrorMessage

Used when API request fails.

Suggested props:

```
type ErrorMessageProps = {  
  message: string  
}
```

EmptyState

Used when a list is valid but empty.

Suggested props:

```
type EmptyStateProps = {  
  title: string  
  description?: string  
}
```

Example:

```
<EmptyState  
  title="No agents found"  
  description="Create your first agent to get started."  
/>
```

8. Import conventions

Try to keep imports consistent.

Preferred if alias is configured:

```
import api from "@services/api"  
import { isAdmin } from "@services/authHelpers"  
import LoadingSpinner from "@components/common/LoadingSpinner"
```

If alias is not configured yet, use relative imports temporarily.

Avoid duplicate helper logic inside UI components.

9. Checkpoints before marking layout/UI task complete

Protected layout

- ☐ Header visible on protected pages.
- ☐ Sidebar visible on protected pages.
- ☐ Public pages like `/login` do not show sidebar/header.
- ☐ Main content renders through `<Outlet />`.
- ☐ Layout works on browser width changes.
- ☐ No console errors.

Header

- ☐ Shows logged username.
- ☐ Shows logout button.
- ☐ Logout clears auth and returns to login.
- ☐ Does not use hardcoded user data.

Sidebar

- ☐ Admin-only links hidden for employees.
- ☐ Navigation works with React Router.
- ☐ Active route is visually clear.
- ☐ Sidebar does not break protected route logic.

Common UI states

- ☐ `LoadingSpinner` created.
- ☐ `ErrorMessage` created.
- ☐ `EmptyState` created.
- ☐ Components are reusable and not tied to one page.

10. What not to touch without coordination

Please avoid changing these files without coordinating first:

```
src/services/api.ts
src/services/authService.ts
src/services/authHelpers.ts
src/services/agentService.ts
src/routes/ProtectedRoute.tsx
src/routes/AdminRoute.tsx
```

These files are part of the current authentication/API implementation and are being used as the base for the next backend integration tasks.

11. Suggested next UI/UX tasks after layout

Once the protected layout is stable:

1. Improve `DashboardPage.tsx` visual structure.
2. Add cards for:
 3. logged user
 4. role
 5. my agents count
 6. subordinate agents count for admin
7. Prepare reusable table/list components for future pages.
8. Prepare empty/loading/error states on dashboard.
9. Create consistent page titles and spacing.

The goal is that future API pages can be added quickly without redesigning layout each time.